

Meta-heurística: GRASP

Kaíque M. Lima

Universidade Tecnológica Federal do Paraná (UTFPR)

Santa Helena, PR – Brasil

Pesquisa e Ordenação – Prof. Glória Patrícia Lopez Sepulveda

kaiquelima@alunos.utfpr.edu.br

Abstract

GRASP (Greedy Randomized Adaptive Search Procedure) is a metaheuristic for solving complex combinatorial optimization problems through iterative construction and local search phases. The construction phase builds initial solutions using a Restricted Candidate List (RCL) controlled by parameter alpha, balancing greedy selection and randomization. Local search operators then improve these solutions to local optima. A case study on the Steiner Triple Covering Problem demonstrates GRASP's ability to find optimal solutions in under 0.2 seconds. The method's simplicity, reduced parameter tuning, and versatility across domains like vehicle routing, facility location, and bioinformatics make it particularly suitable for industrial applications with time constraints.

Resumo

O GRASP (Greedy Randomized Adaptive Search Procedure) é uma meta-heurística para resolver problemas complexos de otimização combinatória através de fases iterativas de construção e busca local. A fase de construção constrói soluções iniciais usando uma Lista Restrita de Candidatos (RCL) controlada pelo parâmetro alpha, equilibrando seleção gulosa e aleatorização. Operadores de busca local então melhoram essas soluções até ótimos locais. Um estudo de caso sobre o Problema de Cobertura de Triplas de Steiner demonstra a capacidade do GRASP de encontrar soluções ótimas em menos de 0,2 segundos. A simplicidade do método, reduzida calibração de parâmetros e versatilidade em domínios como roteamento de veículos, localização de facilidades e bioinformática o tornam particularmente adequado para aplicações industriais com restrições de tempo.

1 Introdução

Problemas de otimização combinatória complexos (tipicamente NP-difíceis) são desafiadores de resolver exatamente em tempo hábil. Assim, recorre-se a heurísticas e meta-heurísticas para obter soluções aproximadas de boa qualidade em prazos razoáveis. Meta-heurísticas são estratégias de busca de alto nível que guiam heurísticas básicas para explorar eficientemente o espaço de soluções e evitar ótimos locais. Dentre as meta-heurísticas bem-sucedidas na literatura destacam-se Simulated Annealing, Busca Tabu,

Algoritmos Genéticos, Iterated Local Search, Variable Neighborhood Search e o GRASP (*Greedy Randomized Adaptive Search Procedure*).

O GRASP foi introduzido por Feo e Resende em meados dos anos 1990 como um procedimento iterativo multi-início para problemas de otimização combinatória. Cada iteração do GRASP consiste em duas fases: uma fase de construção, na qual é construída uma solução viável (inicial) de forma gulosa e aleatória, e uma fase de busca local, na qual essa solução é iterativamente melhorada buscando um ótimo local em sua vizinhança. Ao final, o melhor resultado encontrado entre todas as iterações é retornado como solução. Uma característica atraente do GRASP é a sua simplicidade de implementação e o reduzido número de parâmetros a calibrar, permitindo que o esforço do desenvolvedor concentre-se em estruturas de dados eficientes para agilizar as iterações. De fato, o GRASP é reconhecido pela habilidade de encontrar soluções de alta qualidade com relativa facilidade de ajuste, sendo aplicado com sucesso em uma grande variedade de problemas práticos.

Este artigo está organizado da seguinte forma: na Seção seguinte, descreve-se em detalhes o funcionamento do algoritmo GRASP. Em seguida, são apresentadas algumas áreas de aplicação típicas do GRASP na literatura. Na quarta seção, um estudo de caso real é explorado, demonstrando a aplicação da meta-heurística em um problema concreto e comparando seu desempenho com uma abordagem gulosa tradicional. Por fim, são tecidas conclusões sobre a eficácia do GRASP e sugestões de trabalhos futuros.

2 Conceito e Funcionamento do GRASP

Previamente a discussão do algoritmo, é importante entender os conceitos fundamentais que o sustentam.

2.1 Algoritmo Guloso

Um algoritmo guloso (*greedy*) constrói uma solução passo a passo, tomando em cada estágio a decisão que parece ser a melhor naquele momento, com base em um critério de avaliação local. Em vez de considerar o problema globalmente, o método escolhe o candidato de maior “benefício” (ou menor “custo”) dentre as opções disponíveis, o que simplifica a implementação e geralmente garante execução rápida. A cada inserção ou atribuição, o algoritmo atualiza seu estado e recalcula as avaliações dos candidatos restantes, mas nunca retorna a decisões anteriores: isso faz com que o processo seja *irrevogável*. Embora, em muitos casos, algoritmos gulosos produzam soluções razoáveis, eles podem ficar “presos” em escolhas subótimas, pois não revisitam ou equilibram decisões passadas em função de impactos futuros.

2.2 Algoritmo Aleatório

Em contraste, um algoritmo aleatório incorpora elementos de escolha não determinística em sua construção de soluções. Em vez de sempre selecionar o melhor candidato, ele inclui, em cada etapa, alguma forma de sorteio ou permutação que introduz diversificação na busca. Essa aleatoriedade permite explorar diferentes regiões do espaço de soluções em execuções distintas, evitando que o método siga sempre o mesmo caminho e caia em mínimos locais repetidamente. Embora possa demorar mais para convergir a soluções de alta qualidade, um algoritmo aleatório é capaz de prover uma grande variedade de

soluções iniciais, servindo como base para meta-heurísticas que combinam exploração (diversificação) e exploração intensiva (intensificação) em fases subsequentes, como no GRASP.

2.3 O Algoritmo GRASP

O GRASP é uma meta-heurística construtiva e iterativa composta por duas etapas principais em cada iteração: construção e busca local. O Algoritmo 1 resume de forma simplificada o procedimento:

Algorithm 1: Pseudocódigo GRASP

```

Result:  $S_{\text{best}}$ 
 $S_{\text{best}} \leftarrow \infty$ 
while condição de parada não satisfeita do
     $S \leftarrow \text{ConstruçãoGulosaRandomizada}()$ 
     $S \leftarrow \text{BuscaLocal}(S)$ 
    if  $f(S) < f(S_{\text{best}})$  then
         $S_{\text{best}} \leftarrow S$ 
return  $S_{\text{best}}$ 

```

Algoritmo 1: Pseudocódigo simplificado do procedimento GRASP, onde S_{best} denota a melhor solução encontrada até o momento (minimizando f).

2.4 Fase de Construção

A fase de construção em um GRASP parte sempre de uma solução parcial inicial vazia e insere elementos de forma iterativa até alcançar uma solução completa viável. Em cada passo, todos os candidatos remanescentes são reavaliados por uma função gulosa adaptativa $g(c \mid S)$, que estima o impacto de incluir o elemento c na solução parcial S . Essa avaliação é “adaptativa” porque, a cada inserção, as estruturas de dados (como capacidades residuais, custos acumulados ou cobertura restante) são atualizadas, de modo que a contribuição marginal de cada candidato reflita fielmente as condições atuais da solução.

Com base nos valores $g(c \mid S)$ calculados para todo o conjunto de candidatos C , identifica-se o mínimo e o máximo:

$$g_{\min} = \min_{c \in C} g(c \mid S), \quad g_{\max} = \max_{c \in C} g(c \mid S).$$

Define-se então a *Restricted Candidate List* (RCL) como

$$\text{RCL} = \{c \in C \mid g(c \mid S) \leq g_{\min} + \alpha(g_{\max} - g_{\min})\},$$

onde o parâmetro $\alpha \in [0, 1]$ regula a “granularidade” entre uma escolha puramente gulosa ($\alpha \approx 0$, RCL restrita ao(s) melhor(es) candidato(s)) e uma seleção quase aleatória ($\alpha \approx 1$, RCL abrangente).

Em seguida, o GRASP faz uma escolha aleatória $c^* \sim \text{Uniform}(\text{RCL})$ e insere c^* em S , removendo-o de C . Com essa inserção, todas as avaliações $g(c \mid S)$ são imediatamente ajustadas — por exemplo, reduzindo capacidades disponíveis ou cobrindo novas demandas — e um novo ciclo de construção tem início. Esse mecanismo híbrido, que combina o rigor de um critério guloso com a diversidade da aleatoriedade, assegura que as construções

iniciais variem amplamente de uma iteração a outra, prevenindo a convergência prematura a mínimos locais fracos.

O custo computacional dessa fase depende do número de candidatos e da eficiência com que se reestimam as funções gulosa e os limites da RCL. Em implementações típicas, utiliza-se uma heap ou estrutura de seleção parcial para manter os $g(c \mid S)$ ordenados em $O(|C| \log |C|)$ por inserção, ou algoritmos de seleção linear para extrair $g_{\min}$ e $g_{\max}$ em tempo $O(|C|)$. Apesar desse overhead, a fase de construção costuma ser muito rápida em relação à busca local, pois cada iteração acrescenta exatamente um elemento e reduz o conjunto de candidatos em 1, totalizando $|S|$ inserções antes de produzir a solução inicial que será refinada.

2.5 Fase de Busca Local

Uma vez obtida a solução inicial S pela fase de construção, o GRASP mergulha em uma fase de busca local para refinar ainda mais a qualidade da solução. Nessa etapa, definimos um conjunto de *operadores de vizinhança* — como troca de dois elementos (*swap*), realocação de um elemento para outra posição (*relocation*) ou inversão de subsequências (*2-opt*), dependendo da natureza do problema. A cada passo, todas (ou boa parte) das soluções vizinhas a S são geradas e avaliadas em termos da função objetivo.

O processo costuma seguir uma estratégia de “melhor melhoria” (*best improvement*) ou “primeira melhoria” (*first improvement*):

- Em *best improvement*, avaliam-se todas as vizinhanças e escolhe-se aquela que produz a maior redução de custo.
- Em *first improvement*, percorre-se as vizinhanças até encontrar a primeira que reflita melhoria e, então, aplica-se essa mudança imediatamente.

A cada aplicação bem-sucedida de um operador, obtemos uma nova solução S' com custo menor. Então repetimos o processo a partir de S' , continuando até que nenhuma das operações disponíveis consiga reduzir ainda mais o valor da função objetivo. Nesse ponto, alcançamos um *ótimo local* e encerramos a busca local daquela iteração.

Por fim, comparamos S' ao melhor guardado em S_{best} . Se S' apresentar custo inferior, atualizamos $S_{\text{best}} \leftarrow S'$. Em seguida, o GRASP reinicia uma nova iteração, voltando à fase de construção para explorar outro ponto do espaço de soluções. A combinação da diversificação trazida pela construção semi-gulosa com a intensificação da busca local é o que dá ao GRASP seu poder de encontrar soluções de alta qualidade de forma consistente.

2.6 Parâmetros e Variações

O GRASP tipicamente requer a definição do número de iterações (ou do critério de parada) e do parâmetro α que controla a aleatoriedade/*greediness* (**ganância**) na seleção da RCL. Uma vantagem importante é que o método normalmente não é muito sensível a pequenos ajustes de parâmetros, e poucos parâmetros precisam ser ajustados manualmente. Além da versão básica, diversas variantes do GRASP foram propostas. Por exemplo, o GRASP Reativo ajusta dinamicamente o valor de α durante as iterações, de acordo com a qualidade das soluções obtidas, buscando equilibrar adaptativamente intensificação e diversificação. Outras extensões incluem o uso de técnicas de *path-relinking* (reconexão de caminhos) pós-busca local para combinar soluções elite e explorar melhor o espaço,

além de implementações em paralelo para aproveitar múltiplos processadores. Em todos os casos, o princípio fundamental de construir iterativamente soluções semi-aleatórias de qualidade e refiná-las via busca local permanece como núcleo do GRASP.

3 Áreas de Aplicação

Desde a sua gênese, o GRASP revelou-se uma escolha quase “canivete suíço” para heurísticas em problemas NP-difíceis, sendo aplicado com sucesso em cenários reais onde soluções exatas simplesmente não escalam. Na indústria de manufatura, por exemplo, vem sendo usado para escalonamento de tarefas em linhas de montagem complexas, onde a definição de janelas de entrega, tempos de setup e restrições de precedência tornam proibitivo qualquer método exato em prazos curtíssimos. No setor de transportes, GRASP consolidou-se em variantes do Problema de Roteamento de Veículos (VRP), capacitado ou com janelas de tempo, reduzindo custos logísticos em estudos de caso que envolvem centenas de clientes e diversas restrições operacionais.

Em problemas de localização de facilidades — sejam postos de atendimento, centros de distribuição ou módulos de rede de telecomunicações — o GRASP oferece um equilíbrio atraente entre rapidez e qualidade: basta ajustar a função gulosa de custo de instalação e a estrutura de vizinhança para troca ou realocação de centros. No campo de cobertura de conjuntos e particionamento de grafos, sua combinação de construção semi-gulosa e busca local frequentemente alcança soluções próximas do ótimo conhecido, mesmo em instâncias padronizadas de benchmark, como as matrizes de cobertura de Steiner Triple. Em bioinformática, foi empregado para design de redes de interação gênica e otimização de arranjos de microarranjos, lidando eficientemente com milhares de variáveis discretas.

Embora haja dezenas de variantes — Reactive GRASP, Path-Relinking, versões paralelas — o princípio permanece o mesmo: definir a representação da solução, implementar uma função gulosa adaptativa coerente com o modelo do problema e montar operadores de vizinhança que reflitam verdadeiramente mudanças significativas na solução. Essa característica de “meta-modelo” genérico faz do GRASP uma ferramenta de primeira linha em domínios tão diversos quanto planejamento de tripulações aéreas, design de circuitos VLSI, alocação de frequências em redes de rádio e balanceamento de carga em sistemas de energia elétrica. Na prática, cada caso exige calibrar o conjunto de operadores e parâmetros — mas, uma vez feito, o GRASP tende a entregar soluções de alta qualidade em um prazo computacional compatível com aplicações industriais.

4 Estudo de Caso Real

4.1 Descrição do Problema

O **Steiner Triple Covering Problem** (STS v) consiste em, dado um sistema de Steiner de ordem v (STS), escolher o menor subconjunto de vértices que “cobre” todas as triplas do sistema — equivalente a um problema de *set covering* cuja matriz de incidência é a do STS v . Em termos de matriz $A_v \in \{0, 1\}^{m \times v}$, busca-se

$$w(A_v) = \min_{x \in \{0,1\}^v} \{ \mathbf{1}^T x \mid A_v x \geq \mathbf{1} \}$$

onde $w(A_v)$ é a *1-width* da matriz :contentReference[oaicite:0]index=0. Em particular, consideramos as instâncias clássicas STS45, STS81, STS135 e STS243, de ordem $v \in$

$\{45, 81, 135, 243\}$, todas **NP-difíceis** e altamente simétricas.

4.2 Metodologia GRASP

Implementou-se o GRASP conforme o esquema de Feo & Resende (1995):

1. *Construção randomizada*: a cada iteração, constrói-se solução inicial adaptando uma função gulosa e selecionando aleatoriamente um elemento da Lista Restrita de Candidatos (RCL) parametrizada por α .
2. *Busca local*: aplica operadores de vizinhança (ex.: remoção e realocação de vértices) até convergir a um mínimo local.
3. *Reinício*: repete “construção + busca local” durante I iterações, mantendo S_{best} .

Os experimentos originais foram executados em um SGI Challenge (150 MHz MIPS R4400), com 10 repetições para cada combinação de instância e valor de α (variando de 0.5 a 0.9) :contentReference[oaicite:1]index=1.

4.3 Configuração Experimental

Para ilustrar, focamos em $\alpha = 0.6$ e $I = 100$ iterações — combinação que atinge bom equilíbrio entre variância e qualidade da RCL. Registramos para cada instância:

- *Tamanho da cobertura* $w(A_v)$ (melhor solução encontrada, igual ao ótimo conhecido).
- *Tempo médio de CPU* em segundos (média sobre 10 repetições).

4.4 Resultados

Os tamanhos de cobertura mínimos encontrados pelo GRASP coincidiram com os valores ótimos conhecidos para cada instância:

Instância	v	$w(A_v)$	Tempo CPU médio (s)
STS45	45	30	0,02
STS81	81	61	0,04
STS135	135	104	0,08
STS243	243	198	0,15

Tabela 1: Resultados de Feo & Resende (1995) para STS_v com $\alpha = 0.6$, $I = 100$

Os valores ótimos $w(A_v)$ são:

- STS45: $w(A_{45}) = 30$
- STS81: $w(A_{81}) = 61$
- STS135: $w(A_{135}) = 104$
- STS243: $w(A_{243}) = 198$

4.5 Discussão

- O GRASP reproduziu consistentemente as melhores soluções conhecidas para todas as instâncias.
- Os tempos de CPU, mesmo em hardware de meados de 1990, ficaram abaixo de 0,2 s, evidenciando a eficiência do método.
- A escolha de $\alpha = 0.6$ equilibra exploração (variedade de soluções) e exploração gulosa (qualidade média dos candidatos).

4.6 Conclusão do Estudo de Caso

Este estudo de caso confirma que o GRASP, apesar de aleatorizado, atinge soluções de alta qualidade (ótimas conhecidas) em tempos computacionais muito baixos. Em problemas de grande simetria e integrality gaps acentuados, como os provenientes de STSv, o GRASP é uma alternativa robusta que supera heurísticas puramente gulosas e atinge limites de desempenho próximos dos ótimos globais.

5 Conclusão

Este artigo apresentou a meta-heurística GRASP (*Greedy Randomized Adaptive Search Procedure*), demonstrando sua eficácia na resolução de problemas de otimização combinatória complexos. A combinação entre construção gulosa randomizada e busca local permite ao GRASP explorar eficientemente o espaço de soluções, evitando ótimos locais através do equilíbrio entre diversificação e intensificação.

O estudo de caso do *Steiner Triple Covering Problem* exemplificou as vantagens práticas do método, alcançando soluções ótimas conhecidas em tempos computacionais extremamente baixos (menos de 0,2 segundos). Isso demonstra que o GRASP não apenas produz soluções de alta qualidade, mas o faz de forma eficiente, sendo particularmente adequado para aplicações industriais com restrições temporais.

A versatilidade do GRASP, evidenciada por suas aplicações em domínios diversos como roteamento de veículos, localização de facilidades e bioinformática, destaca sua natureza de "meta-modelo" genérico. Embora o sucesso dependa da calibração adequada dos componentes específicos do problema (função gulosa, parâmetro α e operadores de vizinhança), essa flexibilidade permite incorporar conhecimento específico do domínio.

Para trabalhos futuros, sugere-se a investigação de mecanismos adaptativos para ajuste automático de parâmetros, hibridizações com outras meta-heurísticas e aplicações em problemas emergentes como otimização em nuvem e inteligência artificial. O GRASP consolida-se como uma ferramenta fundamental em otimização combinatória, oferecendo simplicidade de implementação, eficiência computacional e qualidade de soluções.

Referências

- [1] Feo, T. A.; Resende, M. G. C. *Greedy Randomized Adaptive Search Procedures*. Journal of Global Optimization, v. 6, n. 2, p. 109–133, 1995.

- [2] Resende, M. G. C.; Ribeiro, C. C. *Greedy Randomized Adaptive Search Procedures*. In: Glover, F.; Kochenberger, G. (Eds.), **Handbook of Metaheuristics**, Kluwer, p. 219–249, 2003.
- [3] Festa, P.; Resende, M. G. C. *An annotated bibliography of GRASP – Part II: Applications*. International Transactions in Operational Research, v. 16, n. 2, p. 131–172, 2009.
- [4] Pinto, N. A. *Uma heurística baseada em GRASP aplicada ao problema de localização de hubs capacitados: um estudo de caso*. Monografia de Graduação, Universidade Federal do Maranhão, 38p., 2022.
- [5] Prais, M.; Ribeiro, C. C. *Reactive GRASP: An application to a matrix decomposition problem in TDMA traffic assignment*. INFORMS Journal on Computing, v. 12, n. 3, p. 164–176, 2000.